LLM-Based Quest Generation for Role-Playing Games

A Chapter-Structured Approach Using World States and Action Libraries

Niels Weissmann | Student Number: 236814
Breda University of Applied Sciences
Supervisor: Edirlei Soares de Lima
June 8th 2026

Abstract

Role-playing games rely heavily on hand-authored quests to deliver engaging player experiences, yet the manual creation of quest content is resource-intensive and limits the variety and replayability of game worlds. Large language models (LLMs) have shown promise for generating structured game content, and this paper proposes a method for using LLMs to automatically generate quest plans that are logically consistent with the game world. The method structures the game narrative as a sequence of chapters, each composed of one or more quests that advance the overarching story. The game world is represented as a formal state that encodes the current configuration of locations, characters, items, and their conditions, extracted from the game and provided as input to the generation pipeline. Available actions are defined through an explicit action library, and quest goals are specified as natural language descriptions of the situation the chapter should resolve. For each chapter, the LLM generates a candidate action sequence in structured JSON format, the output is validated for structural correctness and retried if malformed, and the resulting quest plan is executed by the player in the game. The method is evaluated on a zombie-survival RPG scenario across three main quest chapters, comparing five LLMs over ten runs each across six metric categories covering structural validity, action validity, world state consistency, inventory consistency, dialogue coverage, and generation performance. Results show that model choice meaningfully affects rule compliance and content quality, and that models generating more content do not consistently produce fewer violations.

Keywords: *quest generation; large language models; procedural content generation; role-playing games*

1. INTRODUCTION

Quest design is one of the most labour-intensive activities in role-playing game (RPG) development [3, 7]. A quest consists of a sequence of tasks that the player must accomplish in order to advance the story or progress through the game world, often involving interactions with characters, collecting items, and overcoming challenges [11]. Producing a varied quest catalogue by hand demands significant designer effort and does not scale well [3, 7], particularly for smaller development teams or games that aim to offer replayable content.

Recent advances in large language models (LLMs) have renewed interest in procedural content generation for games. Prior work has demonstrated their utility for NPC dialogue, narrative generation, and agent behaviour in game-like environments [1, 2]. Quest generation, however, differs from these tasks in an important way: a quest is not merely a piece of text but a structured plan of actions that must be logically consistent with the game world. A step that moves the player into a locked location without a key, or references an item that no longer exists, renders the quest unplayable regardless of narrative quality. Fluent text is therefore a necessary but insufficient condition for usable quest generation.

This paper proposes a method for generating structured quest plans using LLMs. The game world state, which encodes the current configuration of locations, characters, items, and their conditions, is extracted from the game and provided as input to the generation pipeline. An action library defines the valid actions available to the player, constraining the LLM to produce only steps the game engine can execute. Quest goals are specified as natural language descriptions of the situation the chapter should resolve. Generation proceeds chapter by chapter: the LLM produces a candidate action sequence in structured JSON format, the output is validated for structural correctness and retried if malformed, and the resulting quest plan is executed by the player in the game. The goal of this work is to investigate whether LLMs can generate logically consistent quest plans with adequate NPC dialogue coverage when grounded in a formal representation of the game world, and to evaluate how different LLMs perform on this task across a set of defined quality metrics.

The method is evaluated on a zombie-survival scenario across three quest chapters, comparing five LLMs over ten runs each: Qwen3.5-122B-A10B, Qwen3.6-27B, GPT-OSS-120B, Llama-3.3-70B-Instruct, and Claude Opus 4.7. The evaluation covers six metric categories: structural validity, action validity, world state consistency, inventory consistency, dialogue coverage, and generation performance.

The remainder of this paper is structured as follows. Section 2 reviews related work. Section 3 describes the proposed method. Section 4 presents the game prototype. Section 5 covers evaluation and results. Section 6 concludes.

2. RELATED WORK

A. Classical Approaches to Quest Generation

Before the rise of neural language models, procedural quest generation relied primarily on rule-based planners, template systems, and grammars. Doran and Parberry [12] analysed quest structures across four MMORPGs and proposed a classification of quest types, using this classification as the basis for a prototype quest generator. Their approach demonstrates that quest structures share common patterns that can be abstracted and reused, but the output is bounded by the predefined classification and does not adapt to specific game world states.

Lima et al. [3] proposed combining genetic algorithms with automated planning to generate quest structures guided by story arcs, producing results comparable to professionally designed content. Their subsequent work extended this to branching quest generation, maintaining narrative coherence across multiple possible player paths [4]. An earlier contribution addressed nondeterministic quests through a planning, execution, and monitoring architecture that adapts the quest structure as the player progresses [5]. These methods offer formal correctness guarantees by construction: because quest steps are produced by a planner operating over an explicit state space with formally defined operators, the output is guaranteed to be logically valid. The trade-off is that they require exhaustive manual specification of planning operators, preconditions, and goal states, and the resulting content tends to be structurally formulaic. Dormans and Bakkes [6] offered an alternative using shape grammars to separate mission structure from spatial layout, enabling flexible recombination of quest components, though the output remains bounded by the richness of the underlying grammar.

B. LLM-Based Quest and Narrative Generation

Värtinen et al. [7] conducted one of the earliest systematic studies of LLM-based quest generation, using GPT-2 and GPT-3 to produce free-text quest descriptions for RPGs. Their human evaluation found that approximately one in five generated quests was considered acceptable, establishing a useful baseline: LLM-based quest generation is feasible but output quality depends strongly on the model and prompting strategy. Their method produces natural language descriptions only, with no structured output format and no grounding in a specific game world state.

Ashby et al. [8] developed a more architecturally sophisticated approach, combining a hand-crafted knowledge graph with a language model to generate both quests and NPC dialogue. Their system grounded generation in structured representations of the game world, including NPC backgrounds, player history, and lore, and demonstrated that contextual grounding significantly improves coherence. Quest goals are derived implicitly from the player history and the knowledge graph. The output is natural language text intended for presentation to the player, and is not designed to be directly executed by a game engine.

Ranasingha and Arambepola [9] evaluated GPT-3.5, Gemini 1.0, and Claude 2.0 on their ability to generate side quests in structured JSON format, providing each model with comprehensive game context. Their findings confirm that structured JSON output is a viable approach and that model choice meaningfully affects output quality. Their pipeline operates as a single-pass generation without mechanisms for maintaining consistency across multiple quest chapters.

Joshi et al. [2] demonstrated that iterative structured prompting improves agent behaviour in Dungeons and Dragons scenarios, with agents showing higher narrative compliance when given distributed prompts containing shared memories and scenario premises. Their finding that decomposing information across prompt components reduces hallucinations is relevant to the chapter-by-chapter architecture used in the present method. Park et al. [10] introduced generative agents that maintain memory streams and use LLMs to reason about actions in a simulated social environment, sharing structural similarities with the sequential world state representation used in the present method.

C. Positioning of the Present Work

The present method differs from prior LLM-based work in three respects. First, generation is structured around explicit chapters, each with a developer-specified situation description that defines the goal the LLM must resolve. Prior LLM-based approaches either derive goals implicitly from a knowledge graph [8] or provide no explicit goal specification [7], giving the developer less control over narrative direction. Second, available player actions are defined through an explicit action library that constrains the LLM to a vocabulary of actions the game engine can execute directly. Classical planning-based approaches [3, 4, 5] use formal planning operators for a similar purpose, but LLM-based approaches have not adopted this constraint. Third, generation proceeds chapter by chapter with the game world state provided as context for each chapter, enabling the LLM to reason about the current configuration of the world when planning each sequence of steps. Ranasingha and Arambepola [9] also use structured JSON output but treat each quest independently without carrying forward world state context between them.

3. LLM-BASED QUEST GENERATION

The proposed method generates structured quest plans by coordinating a formal game world representation with an LLM through a sequential, chapter-by-chapter pipeline. The method requires four inputs from the developer: a representation of the game world state encoding the current configuration of locations, characters, items, and their conditions, an action library defining the set of valid player actions, a set of world rules specifying the preconditions governing those actions, and a set of chapter descriptions each defining a self-contained quest goal. The pipeline processes each chapter in order, passing the current world state to the LLM as context, collecting the generated action sequence as a quest plan, and passing that plan to the game for the player to execute. As the player progresses through the quest, the game world state is updated naturally through player actions, and the updated state is used as context for generating the next chapter.

A. World State Representation

The game world state encodes the current configuration of all entities relevant to quest generation. It is represented as a structured data object containing the player, a list of characters, a list of locations, a list of items, and a list of interactive objects. Characters are defined by a name, a current location, a list of condition labels such as healthy, infected, or injured, and an inventory of currently held items. Locations are defined by a name, a type such as interior or exterior, and a list of condition labels such as locked, accessible, or powered. Items are defined by a name, a type, and a current location. The location graph is encoded as a list of directional connections between locations, and movement between locations not connected by a declared path is prohibited by the world rules.

The world state is extracted from the game and provided to the generation pipeline as a structured JSON object. The LLM receives this representation as context when generating each chapter, allowing it to reason about the current configuration of the world when planning action sequences. The following example illustrates the JSON format of a world state fragment:

```
{
  "player": { "name": "john", "location": "safehouse",
              "conditions": ["healthy"], "inventory": [] },
  "locations": [
    { "name": "safehouse", "type": "interior",
      "conditions": ["safe"] },
    { "name": "laboratory", "type": "interior",
      "conditions": ["locked", "no_power"] }
  ],
  "items": [
    { "name": "lab_key", "type": "key", "location": "safehouse" }
  ]
}
```

In the example above, the `player` is located at the `safehouse` with no items in inventory. The `laboratory` is marked as `locked` and with no_power, and the `lab_key` item is present at the `safehouse`. These condition labels are used by the world rules, which use them to constrain what actions the LLM can plan. For example, the world rules prohibit the player from moving to the laboratory without first using the `lab_key` to unlock it.

B. Action Library

The action library is defined by the game developer and specifies the complete set of actions available to the player in the game world. Each entry defines an action name, an ordered list of named parameters, and a natural language description that conveys the action's semantics to the LLM. The natural language description is important because it allows the LLM to understand what each action does and when it is appropriate to use it, without requiring the model to infer this from the parameter names alone. The following example illustrates the structure of a single action definition:

```
{
  "action": "MOVE",
  "parameters": ["from", "to"],
  "description": "The player moves from one location to another.
                  Movement is only possible if a path exists
                  between the two locations."
}
```

In the example above, the `MOVE` action defines two parameters, `from` and `to`, representing the origin and destination locations. The natural language description communicates the precondition that a valid path must exist between the two locations, which the LLM uses when planning movement steps in the quest.

C. World Rules

The action library is accompanied by a set of world rules that define the preconditions governing valid action execution. These rules reference the world state conditions to determine whether a planned action is valid given the current configuration of locations, characters, and items. They are included in the prompt provided to the LLM alongside the action library and the current world state, establishing consistency between the constraints communicated to the generator and those enforced by the game engine. The following example illustrates a world rule definition:

```
{
  "world_rules": [
    {
      "id": "WSR_01",
      "actors": ["player", "npc"],
      "rule": "MOVE before acting",
      "description": "The actor must MOVE to a location before performing
                      any action there. An actor cannot act at a location
                      they are not currently at."
    }
  ]
}
```

In the example above, rule `WSR_01` specifies that any actor must first execute a `MOVE` action before performing any other action at a target location. Rules of this kind are applied during validation to detect violations in the generated quest plan.

D. Chapter Specification

Quest goals are specified at the chapter level. Each chapter is defined by a set of metadata fields and a natural language situation description. The scale value, drawn from small, medium, or large, is passed to the LLM as a qualitative scope indicator. The situation field contains a short natural language description of the problem the chapter should resolve, which serves as the goal specification for the LLM.

The LLM uses these inputs to determine the chapter title, which is displayed to the player as the name of the current quest arc, the number of sub-quests, each with its own quest title describing the immediate objective, and how the situation is resolved. This design separates goal specification from action semantics: the developer specifies what problem needs to be solved, and the LLM decides how to solve it through a sequence of actions. The following example illustrates the structure of a chapter specification:

```
{
  "id": "C1",
  "quest_type": "main",
  "required": true,
  "scale": "medium",
  "situation": "The village is in danger. There may be a way to help."
}
```

In the example above, the `situation` description provides no instructions about which characters to involve, which locations to visit, or which actions to use. The LLM is free to interpret the goal and construct a sequence of steps that resolves it in a narratively coherent way. The specification also supports optional side quests through the `quest_type` and `required` fields, which are not used in this project but were retained for future projects.

E. Characters

Characters are defined by the game developer and represent the named NPCs the player can interact with during quest execution. Each character entry specifies an `ID`, a `name`, a `role` such as `antagonist_npc` or `supporting_npc`, a `personality` description, a `goal`, a `behaviour` description, and a `dialogue_style`. These fields are provided to the LLM as context during generation, allowing it to produce dialogue and interactions that are consistent with each character's established traits. The following example illustrates the structure of a character definition:

```
{
  "id": "The Doctor",
  "name": "The Doctor",
  "role": "antagonist_npc",
  "personality": "Brilliant, cold-blooded, and methodical...",
  "goal": "Advance his research as far as possible before anyone can stop him.",
  "behaviour": "Operates independently within the Outpost, avoids direct confrontation with
                the player.",
  "dialogue_style": "Precise, clinical, and cold. Speaks in complete, measured sentences."
}
```

In the example above, the character definition provides the LLM with enough context to generate dialogue and behaviour consistent with the character's role in the narrative.

F. Generation Pipeline

The generation pipeline, shown in figure 1, coordinates the components described in the previous sections to produce a quest plan for each chapter. For each chapter, the pipeline constructs a two-part prompt. The system prompt provides the LLM with the action library and its descriptions, the world rules, and a directive specifying whether the current chapter is the final one in the game. The user prompt supplies the current world state, the chapter specification, and the character definitions including personality descriptions. The LLM is instructed to return a structured JSON object conforming to a fixed output schema and to include no additional prose.

The output schema defines a hierarchical quest structure. At the top level it contains chapter metadata. Below that it contains a list of sub-quests, each with a title describing the immediate player objective. Each sub-quest contains an ordered list of steps. Each step specifies a sequential index that orders the steps globally across all sub-quests within the chapter, an action name drawn from the action library, the actor performing the action, which in the generated quest plan is always the player, and a parameters object whose fields correspond to the parameters defined for that action in the action library. Dialogue exchanges between the player and characters are represented as a parameter containing an ordered list of speaker and line pairs. The following example illustrates the full output structure with one element per section:

```
{
  "chapter_id": "C1",
  "chapter_title": "Embers of the Village",
  "quest_type": "main",
  "required": true,
  "scale": "medium",
  "quests": [
    {
      "quest_id": "C1Q1",
      "quest_title": "Word from the Store",
      "steps": [
        {
          "step": 1,
          "action": "MOVE",
          "actor": "player",
          "parameters": { "from": "safehouse", "to": "village" }
        },
        {
          "step": 3,
          "action": "TALK",
          "actor": "player",
          "parameters": {
            "target": "anne",
            "place": "store",
            "dialogue_content": [
              { "speaker": "PLAYER", "line": "Anne. The village is falling apart." },
              { "speaker": "anne", "line": "There's talk. Sarah was working on a cure." },
              ...
            ]
          }
        },
        ...
      ]
    },
    ...
  ]
}
```

In the example above, the chapter contains a list of sub-quests, each with an ordered list of steps. Each step references an action from the action library and includes the corresponding parameters. Dialogue exchanges are embedded inline as an ordered list of speaker and line pairs within the relevant step's parameters.

Reasoning tokens enclosed in model-specific tags are stripped from the response before JSON parsing is attempted. If the LLM returns a response that cannot be parsed as valid JSON, the generation call is retried with the same prompt up to three times. If the parsed output contains dialogue exchanges that are missing or malformed, a targeted dialogue validation step requests only the missing dialogue content and patches it into the output before accepting it. Once a valid chapter plan is produced it is passed to the game for the player to execute. As the player completes the steps, the game world state is updated and the updated state is provided as context for the next chapter. The full system prompt used in this work is available in the accompanying code repository [18].

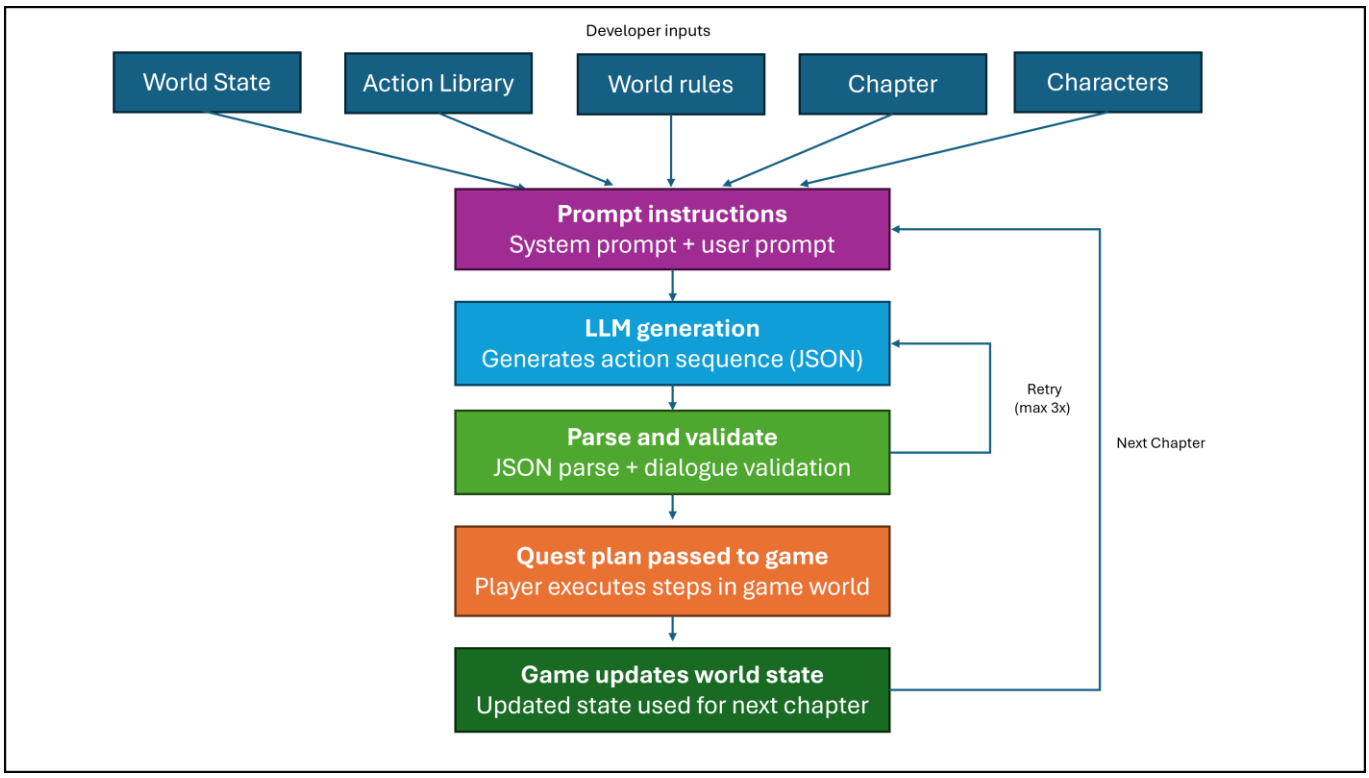

Figure 1. Architecture of the quest generation pipeline.

4. GAME PROTOTYPE

To validate the proposed method in an actual game context, it was integrated into a 2D top-down RPG built using the LÖVE2D framework in Lua, adapted from a game prototype developed in prior work [5]. The game is set on a zombie-infested island and follows a single player-controlled character navigating a world populated by eight NPCs: Anne, George, Sarah, Marcus, Viktor, Linda, Mary, and the Doctor. The game world comprises fourteen locations connected by a directional path graph, including interior locations such as a safehouse, laboratory, hospital, store, and outposts, and exterior locations such as a village, farm, dock, graveyard, and power station. Gameplay involves navigating between locations, shooting zombies and collecting ammunition, collecting and managing items, interacting with NPCs through dialogue, and resolving quest objectives.

A. Quest Integration

The generation pipeline is exposed as a REST API implemented as a standalone Python service. This separation allows the LÖVE2D game client, written in Lua, to offload all LLM interaction to the Python service without requiring any LLM-specific logic in the game itself. It also allows the underlying model to be swapped out by changing only the service, without modifying the game code. The game communicates with this service at runtime by sending a request containing the chapter specification, the current world state, the action library, the world rules, and the character definitions, and receives a quest plan in JSON format in return.

Figure 2 illustrates the architecture of the integration between the game client and the generation service. The Quest Manager component is responsible for initiating generation requests to the service and passing the returned quest plan to the Quest Runner. The Quest Runner tracks player progress through the generated steps and checks completion conditions as the player navigates the world, collects items, and completes interactions. For dialogue steps, the Quest Runner delegates to the Dialogue interface component, which displays the LLM-generated exchange to the player. For all other action types such as `MOVE`, `PICKUP`, or `UNLOCK`, the Quest Runner handles completion checking directly without involving the Dialogue interface. The Quest interface component displays the current objectives to the player and updates as sub-quests are completed. The World State component maintains the live game world state and is read by the generation service when constructing each chapter prompt.

At startup, the game requests generation of the first chapter while displaying a loading screen to the player. The game waits for the quest plan to be returned before proceeding, ensuring the plan is available before gameplay begins. Once received, the quest plan is handed to the Quest Runner. If generation fails during the loading phase, the player is offered the option to request a new attempt or proceed without a generated quest, in which case the game runs without quest objectives.

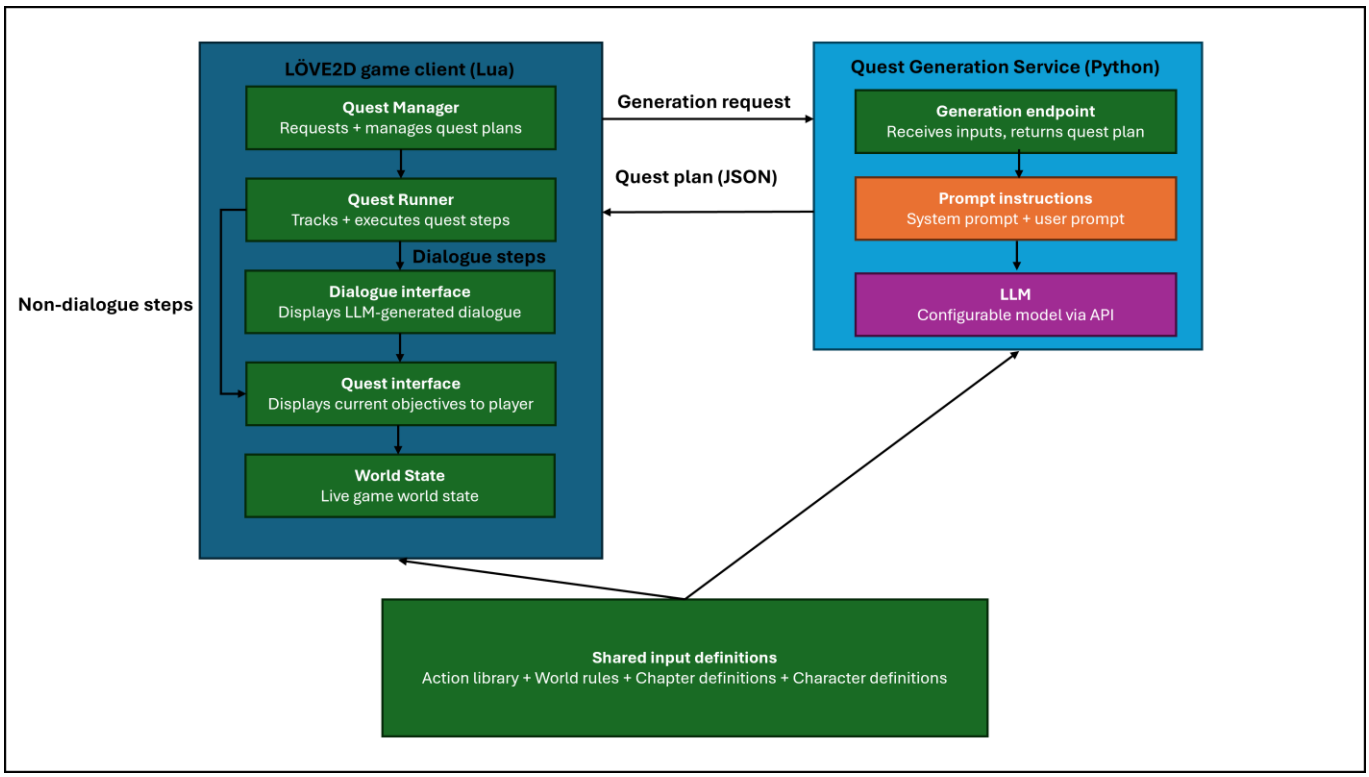

Figure 2. Architecture of the game prototype and its integration with the quest generation service.

B. Quest Execution

The quest management system reads the active quest plan and monitors player actions against the expected sequence of steps. As the player navigates the game world, completes interactions, and collects items, the system advances through the steps accordingly.

When a dialogue interaction is scheduled in the quest plan, the Quest Runner delegates to the Dialogue interface component, which displays the LLM-generated exchange to the player, as shown in Figure 3(a). The current quest objectives are displayed to the player through the Quest interface component, which updates as earlier sub-quests are completed, as shown in Figure 3(b).

(*a*)

(*b*)

Figure 3. (*a*) Generated dialogue exchange and quest objectives panel displayed during gameplay. (*b*) Current goal display and player inventory during gameplay.

C. Game Setup

The game prototype used to evaluate the method implements the action library, world rules, and chapter specifications described in Section 3 as follows. The action library contains eleven player-executable actions covering movement, item handling, character interaction, access control, and world manipulation: `MOVE`, `PICKUP`, `DROP`, `GIVE_ITEM`, `TALK`, `UNLOCK`, `REPAIR`, `TURN_POWER_ON`, `FORTIFY`, `SYNTHESIZE_CURE`, and `USE_CURE`. The world rules contain sixteen entries governing preconditions for these actions, including movement constraints between connected locations, lock and key mechanics requiring the player to hold the correct key item before entering a locked location, inventory requirements for crafting and repair actions, co-location requirements for character interactions, and dialogue prerequisites before transferring items to characters.

The game world is initialised with the player located at the safehouse. Three main quest chapters are defined. Chapter C1 presents the situation "The village is in danger. There may be a way to help." Chapter C2 presents "The power station has gone dark. The laboratory cannot function without it." Chapter C3 presents "A survivor is missing. Nobody has heard from him in days." Each chapter is marked as required and assigned a medium scale. The step count range corresponding to medium scale, ten to thirty steps, is defined in the evaluation script and used to measure scale mismatch.

The full action library, world rules, and chapter specifications used in this evaluation are available in the accompanying code repository [18].

5. EVALUATION AND RESULTS

A. Methodology

To evaluate the proposed quest generation method, a mixed-methods approach was used combining quantitative metric-based assessment and qualitative review of generated outputs. The evaluation was conducted independently of the game environment, focusing on the quality of the generated quest plans rather than the gameplay experience. Five LLMs were evaluated to assess how model choice affects output quality, rule compliance, and generation performance. The pipeline was run without any correction mechanism, allowing failure modes to surface and be compared directly across models in their unmodified output. The evaluation script and all generated outputs are available in the accompanying code repository [18].

Table 1 summarises the five models evaluated: Qwen3.5-122B-A10B [13], Qwen3.6-27B [14], GPT-OSS-120B [15], Llama-3.3-70B-Instruct [16], and Claude Opus 4.7 [17]. The four locally hosted models were deployed on a shared inference server equipped with an AMD EPYC 9555 64-core processor, 1.5TB of RAM, and four NVIDIA RTX PRO 6000 Blackwell GPUs. Claude Opus 4.7 was accessed via the Anthropic API.

Each model was run 10 times across the 3 chapters defined for the game prototype described in Section 4. Table 2 presents the chapter definitions used in the evaluation.

Table 1. Evaluated model details.

Model	Provider	Type	Total parameters	Active parameters
Qwen3.5-122B-A10B	Alibaba Cloud	Mixture of Experts	122B	10B
Qwen3.6-27B	Alibaba Cloud	Dense	27B	27B
GPT-OSS-120B	OpenAI	Mixture of Experts	117B	~5B
Llama-3.3-70B-Instruct	Meta	Dense	70B	70B
Claude Opus 4.7	Anthropic	Unknown	Unknown	Unknown

Table 2. Chapter definitions used in the evaluation.

Chapter	Quest type	Scale	Situation description
C1	Main quest	Medium	The village is in danger. There may be a way to help.
C2	Main quest	Medium	The power station has gone dark. The laboratory cannot function without it.
C3	Main quest	Medium	A survivor is missing. Nobody has heard from him in days.

The chapters were processed sequentially in each run. To simulate the world state evolution between chapters as it would occur during actual gameplay, a deterministic script replayed the generated action steps after each chapter and updated the world state before passing it as input to the next chapter. This approach approximates the sequential context the game would provide at runtime, allowing multi-chapter consistency to be measured without requiring a human player to execute the quest. Each generation attempt was retried up to three times if the output could not be parsed as valid JSON. Temperature was set to 0.7 across all models, providing a balance between output diversity and adherence to the prompt constraints. Output was capped at 8192 tokens per call, matching the configuration used in the game integration described in Section 4.

Six metric categories were defined to assess different aspects of output quality, described in Table 3. Structural validity measures whether the output conforms to the expected format and scale; the scale mismatch metric specifically measures deviation from an evaluator-defined step count range that was not communicated to the model in the prompt, and reflects observed output size relative to the target scope rather than a rule the model was expected to follow. Action validity measures whether the generated actions are legal and correctly formed. World state consistency measures whether steps respect the current configuration of the game world. Inventory consistency measures whether the model correctly tracks item availability when planning multi-step sequences. Dialogue coverage measures whether quests include sufficient NPC interaction. Performance measures generation efficiency in terms of time and token usage.

Table 3. Evaluation metrics and descriptions.

Category	Metric	Description
Structural validity		
	JSON parse success	Whether the LLM output could be parsed as valid JSON
	Number of quests	Number of sub-quests generated per chapter
	Number of steps	Total action steps generated per chapter
	Scale mismatch	Steps fall outside the target range of 10–30 steps for the assigned medium scale
Action validity		
	Invalid actions	Actions not present in the action library
	Actor not player	Steps where the actor field is not set to player
	Missing parameters	Required parameters absent from a generated step
	Unknown parameters	Parameters present that do not belong to that action
World state consistency		
	Locked move violation	Player moved into a locked location without a prior unlock step
	Move no-op	Player moved from a location to the same location
	Hallucinated locations	Locations referenced that do not exist in the world state
	Hallucinated NPCs	Characters referenced that do not exist in the world state
	Hallucinated items	Items referenced that do not exist in the world state
	Pickup without prior move	Item picked up without the player first moving to its location
Inventory consistency		
	Synthesis without sample	Cure synthesized without a sample in the player inventory
	Repair without toolkit	Repair performed without a toolkit in the player inventory
	Repair without wood	Repair performed without wood in the player inventory
	Cure used without synthesis	Antidote used before being synthesized
Dialogue coverage		
	Quest without dialogue	Sub-quests containing no NPC interaction steps
	Duplicate dialogue	Two or more consecutive TALK actions with the same NPC at the same location with no intervening action steps
Performance		
	Generation time (s)	Wall-clock time in seconds to generate one chapter
	Input tokens	Number of tokens in the prompt sent to the model
	Output tokens	Number of tokens in the model response
	Attempts	Generation calls before a parseable response was received

The qualitative review was conducted by a single reviewer reading two randomly selected quest outputs per model across all five models. Three aspects of content quality were assessed. Character voice and dialogue tone were evaluated by reading dialogue exchanges and assessing whether each character's speech matched their defined personality and role. Narrative creativity and variety were assessed by examining whether quest structures, titles, and resolutions repeated across runs for the same model, or whether the model found different approaches to resolving the same situation. NPC coverage was assessed by identifying whether models made use of the full cast of available characters or tended to underuse characters by leaving them absent or reducing them to minimal interactions despite having detailed character definitions and having potential value for the quests. These observations are subjective and should be treated as indicative rather than conclusive. To confirm patterns observed during manual reading, the full set of generated outputs was additionally reviewed using Claude (Anthropic) as an LLM assistant to identify recurring themes across all runs. To guide the review and identify areas of interest, measurable proxies were computed

from the generated outputs: total `TALK` steps and dialogue lines per model, average lines per dialogue exchange, unique chapter and quest title counts as a proxy for narrative variety, and NPC interaction counts as a proxy for character coverage. These proxies are reported in the qualitative results section and used to direct attention toward notable patterns rather than to replace the subjective assessment.

B. Quantitative Results

Tables 4 and 5 present the mean values per chapter across all models and metric categories.

All five models produced parseable JSON output in every run. On structural validity, as shown in Table 4, Claude Opus 4.7 and Qwen3.5-122B-A10B generated the most content per chapter, averaging 4.2 and 3.9 sub-quests and 27.6 and 24.4 steps respectively. Claude Opus 4.7 most reliably stayed within the target range of 10–30 steps with a mismatch rate of 0.067, while GPT-OSS-120B and Llama-3.3-70B-Instruct both reached 0.667.

On action validity, all models performed well. Invalid actions were near zero across all models. Qwen3.6-27B produced a mean of 1.067 unknown parameters per chapter with a high standard deviation of 4.456, indicating rare but severe bursts rather than a consistent pattern.

On world state consistency, as shown in Table 4, the locked move violation metric showed the clearest differentiation between models. Claude Opus 4.7 produced zero violations across all 30 chapters. Qwen3.5-122B-A10B averaged 0.767 violations per chapter, Llama-3.3-70B-Instruct averaged 0.6, GPT-OSS-120B averaged 0.2, and Qwen3.6-27B averaged 0.133. Two distinct failure patterns were observed. In the first, the model plans an unlock step but immediately precedes it with a move into the locked location, indicating it understands the location is locked but misapplies the rule that unlocking already moves the player inside. In the second, no unlock step is planned at all, indicating the locked condition is ignored entirely. Qwen3.6-27B, GPT-OSS-120B, and Llama-3.3-70B-Instruct produced exclusively the first type. Qwen3.5-122B-A10B produced both types, with 30 percent of its violations being the second type, all targeting the same location across chapters C2 and C3. GPT-OSS-120B was the only model to hallucinate NPC names, averaging 1.433 invented characters per chapter.

On inventory consistency, as shown in Table 5, all metrics were near zero across all models, indicating that multi-step sequences such as collecting a cure sample, synthesizing an antidote, and using it on a character were handled reliably. On dialogue coverage, quest without dialogue was highest for Qwen3.5-122B-A10B at 0.700 per chapter, meaning a significant proportion of its sub-quests contained no NPC interaction. GPT-OSS-120B produced zero quests without dialogue. Claude Opus 4.7 and Qwen3.6-27B performed well at 0.167 and 0.083 respectively.

Table 4. Mean values per chapter: Structural validity, Action validity, World state consistency (10 runs x 3 chapters).

Metric	Qwen3.5-122B-A10B	Qwen3.6-27B	GPT-OSS-120B	Llama-3.3-70B-Instruct	Claude Opus 4.7
Structural validity					
JSON parse success	1.000	1.000	1.000	1.000	1.000
Number of quests	3.867	2.450	1.800	2.633	4.167
Number of steps	24.433	20.017	16.267	14.167	27.567
Scale mismatch	0.400	0.467	0.667	0.667	0.067
Action validity					
Invalid actions	0.000	0.000	0.033	0.000	0.000
Actor not player	0.000	0.000	0.000	0.000	0.033
Missing parameters	0.000	0.000	0.000	0.000	0.000
Unknown parameters	0.000	1.067	0.000	0.000	0.067
World state consistency					
Locked move violation	0.767	0.133	0.200	0.600	0.000
Move no-op	0.000	0.117	0.200	0.300	0.100
Hallucinated locations	0.000	0.000	0.000	0.000	0.000
Hallucinated NPCs	0.000	0.000	1.433	0.000	0.000
Hallucinated items	0.000	0.000	0.000	0.000	0.000
Pickup without prior move	0.233	0.067	0.033	0.100	0.067

Table 5. Mean values per chapter: Inventory consistency, Dialogue coverage, Performance (10 runs x 3 chapters)

Metric	Qwen3.5-122B-A10B	Qwen3.6-27B	GPT-OSS-120B	Llama-3.3-70B-Instruct	Claude Opus 4.7
Inventory consistency					
Synthesis without sample	0.033	0.033	0.000	0.033	0.000
Repair without toolkit	0.033	0.017	0.033	0.000	0.033
Repair without wood	0.033	0.000	0.000	0.000	0.000
Cure used without synthesis	0.000	0.067	0.000	0.033	0.000
Dialogue coverage					
Quest without dialogue	0.700	0.083	0.000	0.233	0.167
Duplicate dialogue	0.000	0.017	0.000	0.000	0.000
Performance					
Generation time (s)	20.589	53.002	13.994	45.939	52.525
Input tokens	8564.967	8560.100	7899.467	9209.200	11646.900
Output tokens	2555.300	1951.717	2228.533	1477.767	3868.767
Attempts	1.000	1.000	1.000	1.167	1.033

To assess whether observed differences between models are statistically significant, Kruskal-Wallis tests were run on all metrics. All thirty observations per model were used, with the three chapters pooled. The three chapters within a run are not independent, as the evaluation script feeds the updated world state from one chapter as input to the next; treating the 30 pooled values as fully independent therefore overstates the real amount of information. This within-run dependence is a limitation of the test, and results should be treated as exploratory. Multiple comparisons were corrected within each post-hoc test using Holm correction but not across the omnibus tests; as all significant results fall below $p < 0.001$, this does not affect the conclusions.

Dunn's post-hoc tests with Holm correction identified the specific model pairs driving these differences. For locked move violations, Claude Opus 4.7 produced significantly fewer violations than Qwen3.5-122B-A10B ($p < 0.001$) and Llama-3.3-70B-Instruct ($p < 0.001$), and Qwen3.6-27B produced significantly fewer than both ($p < 0.001$). For scale mismatch, Claude Opus 4.7 differed significantly from GPT-OSS-120B ($p < 0.001$), Llama-3.3-70B-Instruct ($p < 0.001$), and Qwen3.6-27B ($p = 0.003$). For quests without dialogue, Qwen3.5-122B-A10B produced significantly more than GPT-OSS-120B ($p < 0.001$), Qwen3.6-27B ($p < 0.001$), and Claude Opus 4.7 ($p = 0.024$). For hallucinated NPCs, GPT-OSS-120B differed significantly from all other models ($p < 0.001$), confirming it as the sole source of NPC hallucinations. For generation time, GPT-OSS-120B was significantly faster than Qwen3.6-27B ($p < 0.001$), Llama-3.3-70B-Instruct ($p < 0.001$), and Claude Opus 4.7 ($p < 0.001$), and Qwen3.5-122B-A10B was significantly faster than Qwen3.6-27B ($p < 0.001$), Llama-3.3-70B-Instruct ($p = 0.001$), and Claude Opus 4.7 ($p < 0.001$).

Table 6. Kruskal-Wallis test results for all metrics across five models.

Metric	H	p	ε^2
Structural validity			
JSON parse success	—	—	—
Number of quests	59.68	<0.001	0.384
Number of steps	55.97	<0.001	0.358
Scale mismatch	29.31	<0.001	0.175
Action validity			
Invalid actions	5.00	0.287	0.007
Actor not player	5.00	0.287	0.007
Missing parameters	—	—	—
Unknown parameters	7.44	0.114	0.024
World state consistency			
Locked move violation	50.60	<0.001	0.321
Move no-op	12.41	0.015	0.058
Hallucinated locations	—	—	—
Hallucinated NPCs	87.10	<0.001	0.573
Hallucinated items	—	—	—
Pickup without prior move	8.85	0.065	0.033

Metric	H	p	ε^2
Inventory consistency			
Synthesis without sample	2.03	0.730	0.000
Repair without toolkit	1.27	0.866	0.000
Repair without wood	5.00	0.287	0.007
Cure used without synthesis	5.93	0.204	0.013
Dialogue coverage			
Quest without dialogue	24.23	<0.001	0.139
Duplicate dialogue	2.00	0.736	0.000
Performance			
Attempts	15.79	0.003	0.081
Generation time (s)	90.35	<0.001	0.596
Input tokens	122.14	<0.001	0.815
Output tokens	78.69	<0.001	0.515
Total tokens	92.04	<0.001	0.607

Note. — indicates all models produced identical values; test not applicable. df (degrees of freedom) = 4 (five models minus one); n = 30 per model (10 runs × 3 chapters pooled). ε^2 = epsilon-squared effect size; values below 0.001 shown as 0.000.

C. Performance

GPT-OSS-120B was the fastest model at 14.0 seconds per chapter on average. Qwen3.5-122B-A10B averaged 20.6 seconds despite its larger total parameter count, faster than Qwen3.6-27B at 53.0 seconds and Claude Opus 4.7 at 52.5 seconds. The speed difference between Qwen3.5-122B-A10B and Qwen3.6-27B is explained by their architectures: Qwen3.5-122B-A10B activates only 10 billion parameters per forward pass as a Mixture of Experts model, while Qwen3.6-27B activates all 27 billion parameters for every token generated.

Claude Opus 4.7 generated the most output tokens per chapter at a mean of 3869, compared to 1478 for Llama-3.3-70B-Instruct. Input token counts varied across models, ranging from 7899 to 11647 per chapter. The higher input token counts for Claude Opus 4.7 are partly explained by its larger output volumes, since the generated quest plan is passed as context for subsequent chapters and models that generate more tokens per chapter also contribute to larger inputs in later chapters. Based on Anthropic's published pricing at the time of evaluation,1 the estimated cost for running Claude Opus 4.7 across 30 chapters was approximately $4.56, or $0.15 per chapter, calculated from the recorded input and output token counts. The four locally hosted models were run on shared institutional infrastructure whose energy and hardware costs are not calculated here but should be considered when comparing deployment options.

D. Qualitative Results

Figure 4 presents two measurable proxies computed from the generated outputs: NPC interaction counts across all runs and dialogue length distributions per model. These proxies were used to identify patterns of interest before reading the outputs, such as which characters were frequently absent, which models produced longer exchanges, and how dialogue volume varied across models. They do not directly measure quality, since longer dialogue does not automatically indicate stronger character voice, broader NPC usage does not guarantee meaningful interactions, and higher quest counts do not mean greater narrative variety. The subjective observations reported below are grounded in these patterns but based on reading the actual generated outputs.

[1] https://www.anthropic.com/pricing

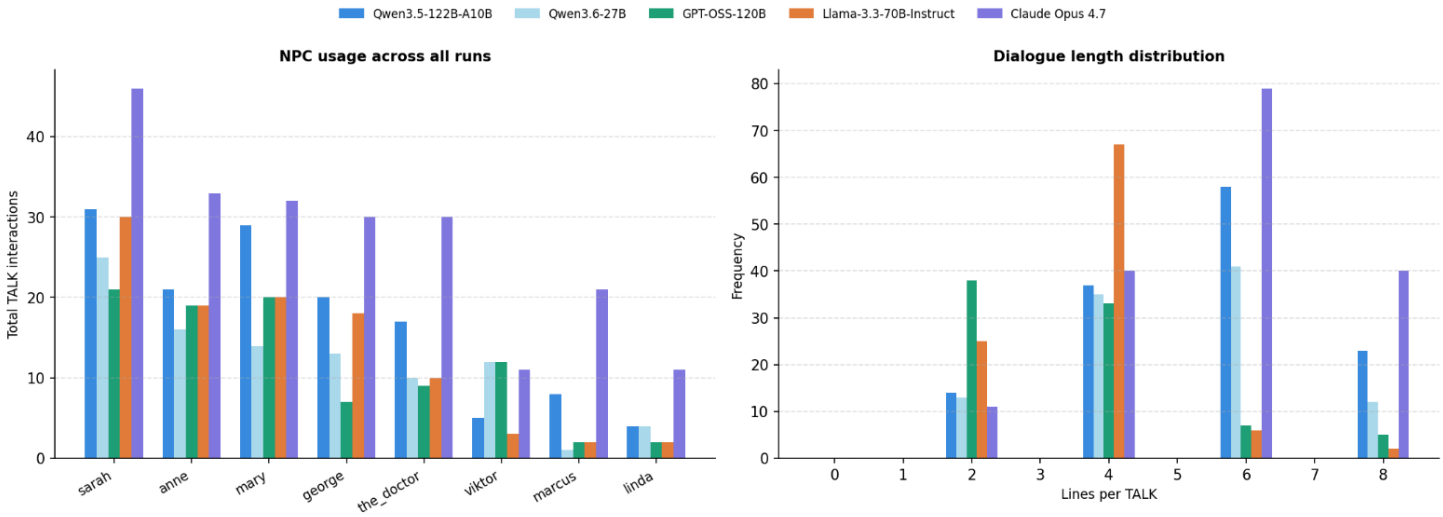

Figure 4. NPC usage across all runs (left) and dialogue length distribution per model (right).

On character voice and dialogue tone, Claude Opus 4.7 produced the most varied exchanges, with character-specific speech patterns and emotional subtext extending beyond the immediate quest objective. Qwen3.5-122B-A10B produced similarly character-driven dialogue among the locally hosted models, with distinct voices for antagonist characters. Qwen3.6-27B maintained individual character identity across runs despite being a smaller model. GPT-OSS-120B produced shorter, more functional exchanges with less tonal variation between characters. Llama-3.3-70B-Instruct produced the most generic dialogue, with exchanges following simple question-answer patterns and little character voice. Antagonist characters were largely absent from Llama-3.3-70B-Instruct outputs despite being narratively central in later chapters, with Viktor and the Doctor rarely appearing even in chapters where they are directly relevant to the story.

To illustrate the qualitative differences observed, the following examples show dialogue generated by three models for the same character. The first example is from Claude Opus 4.7, Chapter C2, in which the player confronts the Doctor:

```
The Doctor: Ah. The variable I have been hearing about. Please — try not to touch anything.
Player: You made the strain. Didn't you.
The Doctor: I refined it. There is a difference. The base organism is naturally occurring. I
merely... improved its transmissibility. Its persistence.
Player: Improved. You killed half the island.
The Doctor: I documented half the island. The research will outlive every one of you.
Including me. That is what matters.
```

In the example above, The Doctor speaks with ideological detachment and moral indifference. The exchange conveys character voice, subtext, and narrative tension without stating the character's nature directly. Claude Opus 4.7 consistently generated longer and more varied exchanges of this kind across the reviewed outputs.

The second example is from Qwen3.5-122B-A10B, Chapter C2, covering the same character:

```
Player: I know you're behind the chaos on this island. Viktor is your puppet, isn't he?
The Doctor: Puppet? Such a crude metaphor. Viktor is a variable I have optimized for control.
He handles the brute force while I handle the science.
Player: You're using the outbreak to test your theories. People are dying for your data.
The Doctor: Data requires a sample size, and tragedy is the most efficient catalyst for
evolution. Viktor's security ensures my work isn't interrupted by... sentimentality.
Player: I'm not leaving until I stop you. Where are you hiding your research?
The Doctor: My research is everywhere, and nowhere. But if you wish to see the source, you'll
find the tools you need in this very room. Take the toolkit — it is the only thing I have no
use for right now.
```

In the example above, The Doctor maintains a similar ideological detachment to the Claude output but frames the relationship with Viktor differently, presenting him as a controlled variable rather than a tool. The exchange demonstrates that Qwen3.5-122B-A10B can produce character-consistent dialogue with subtext, though the framing is more explicit than the Claude example.

The third example is from Llama-3.3-70B-Instruct, Chapter C2, covering the same character:

```
Player: Doctor, I've been investigating the strange happenings on the island. What do you know
about it?
The Doctor: I'm just trying to advance my research. I don't know what you're talking about.
Player: I've seen you working with Viktor. What's your relationship with him?
The Doctor: Viktor is just a means to an end. He provides me with the resources I need to
continue my research.
```

The exchange presented in the example above conveys the same basic information but uses a generic question-answer structure with no distinctive character voice. The Doctor's response is functionally correct but offers no personality beyond the minimum required to advance the quest.

On narrative creativity, Qwen3.5-122B-A10B occasionally deviated from the most obvious resolution path, introducing alternative approaches not present in other models. Claude Opus 4.7 produced the most elaborate quest structures but defaulted to fixed narrative framings in later chapters. Llama-3.3-70B-Instruct showed the least creative range, with later chapters following near-identical structures across runs with little variation in how the situation was resolved. On NPC coverage, as shown in Figure 4, Claude Opus 4.7 and the two Qwen models made use of the full cast of characters across their runs, while GPT-OSS-120B and Llama-3.3-70B-Instruct each left at least one character unused entirely. Claude Opus 4.7 consistently produced the longest and most varied exchanges, while Llama-3.3-70B-Instruct tended toward brief, transactional interactions that covered the minimum required to advance the quest.

E. Summary

Across five models a consistent pattern emerges: models that generate more content tend to produce more rule violations, with one exception. Claude Opus 4.7 generates the most content of any model while also producing the fewest violations, achieving zero locked move violations, the lowest scale mismatch rate, and zero hallucinated characters. Among the locally hosted models, Qwen3.5-122B-A10B generates the most content and variety but is held back by locked move violations and a high rate of sub-quests with no NPC interaction. Qwen3.6-27B is the most rule-compliant locally hosted model at the cost of lower content volume and slower generation. GPT-OSS-120B is let down by hallucinated characters as its primary weakness. Llama-3.3-70B-Instruct is consistent but produces the least creative output of the five models. These results suggest that LLM-based quest generation with structured world state grounding is a viable approach to reducing the manual effort of quest design, with model choice being a significant practical consideration for developers.

6. CONCLUDING REMARKS

This paper presented a method for generating structured quest plans for role-playing games using large language models. The method grounds generation in a formal representation of the game world state, an action library defining the valid actions available to the player, and natural language situation descriptions that specify what each chapter should resolve. Generation proceeds chapter by chapter, with the current world state provided as context for each generation call and updated by the game as the player executes the quest. The evaluation demonstrates that structured world state grounding is a viable approach to LLM-based quest generation and that model choice is a significant practical consideration for developers. All materials including the generation pipeline, evaluation scripts, and input specifications are available in the accompanying code repository [18].

The evaluation results show that all five models can reliably produce parseable, structurally valid quest plans. However, rule compliance varies considerably across models, and the failure modes observed have direct implications for gameplay. Locked move violations are the most consequential failure: when the generated quest plan instructs the player to enter a locked location without first obtaining the key, the player becomes stuck with no objective pointing them toward the solution. A player without prior knowledge of the game world would have no way to know where the key is located, since the quest system no longer guides them. This type of error does not break the game technically but breaks the player experience by making the quest unsolvable without external knowledge.

Generation latency is a practical concern that the evaluation does not fully address. In the current implementation, the first chapter is generated during a loading screen before gameplay begins. Generation time ranged from approximately 14 seconds for GPT-OSS-120B to over 50 seconds for Claude Opus 4.7 and Qwen3.6-27B. For a small prototype this is acceptable, but for a commercial game with higher player expectations the latency of the slower models would likely be perceived as disruptive. GPT-OSS-120B at 14 seconds per chapter is the most viable option on this dimension, though its NPC hallucination issue remains a concern.

Cost scalability is a critical limitation for API-based models in commercial contexts. At the token volumes recorded in this evaluation, Claude Opus 4.7 costs approximately $0.46 per player per three-chapter playthrough. For a game with 10,000 players, that amounts to roughly $4,600 per playthrough cycle, and costs would scale linearly with player count and chapter count. For a game with millions of players or a longer quest structure, API costs would become prohibitive. Locally hosted models avoid API costs entirely, though this shifts the burden to hardware procurement and energy consumption, which carry their own scalability concerns, making them a more realistic option for commercial deployment despite their quality trade-offs. The scale mismatch metric used in the evaluation deserves honest discussion as a limitation of the study design. The step count range used to define scale was not communicated to the models in the prompt, meaning models were assessed against an expectation they had no

means of satisfying directly. This limits the conclusions that can be drawn from that metric and would be straightforwardly addressable by including explicit step count targets in the prompt in future work.

The use of Claude as an LLM assistant to identify recurring themes in the qualitative review introduces a circularity concern: an LLM evaluating LLM-generated content may share biases or blind spots with the models being assessed, potentially missing systematic failure patterns that a human reviewer would catch. The Claude-assisted review should therefore be interpreted as a complement to the manual reading rather than an independent confirmation of it.

Several directions for future work follow from the observed limitations. The two most consistent failure modes, locked move violations and quests without NPC interaction, are likely addressable through targeted prompt engineering rather than model changes. Resolving them would substantially improve the practical usability of the method for all models. A runtime validation step that would check generated plans against the live world state and triggers replanning when violations are detected would reduce the impact of rule violations on player experience without requiring prompt-level fixes. A player study is needed to assess whether generated quests are perceived as coherent, engaging, and varied, since the quantitative metrics used here do not capture the player experience directly. The current method generates only main quest chapters with a fixed sequential structure. Extending it to support side-quests and optional objectives would increase narrative variety and replayability, allowing players to engage with the game world beyond the critical path. This introduces several design challenges: determining when side-quests become available relative to main quest progression, defining how optional world state changes from side-quests interact with the main quest state without causing inconsistencies, specifying how side-quest completion or failure should affect subsequent main quest generation, and integrating side-quest tracking into the existing quest management system alongside the main chapter sequence. Finally, evaluating the method on different game worlds, quest types, and genres would help establish how well the approach generalises beyond the single scenario used in this work.

References

[1] S. Kostilainen, "Next generation of NPC dialogue: Creating responsive NPCs with retrieval-augmented generation and real-time player data," Master's thesis, Jamk University of Applied Sciences, 2024. [Online]. Available: https://urn.fi/URN:NBN:fi:amk-2024053119434

[2] V. Joshi, N. Ajmeri, and K. O'Hara, "Towards LLM-agents that play Dungeons & Dragons using iterative prompting," AI4HGI Workshop, ECAI 2025. [Online]. Available: https://ceur-ws.org/Vol-4097/paper7.pdf

[3] E. S. de Lima, B. Feijó, and A. L. Furtado, "Procedural generation of quests for games using genetic algorithms and automated planning," in Proc. XVIII Brazilian Symp. Computer Games and Digital Entertainment (SBGames 2019), pp. 495-504, 2019. DOI: 10.1109/SBGames.2019.00028

[4] E. S. de Lima, B. Feijó, and A. L. Furtado, "Procedural generation of branching quests for games," Entertainment Computing, vol. 43, 100491, 2022. DOI: 10.1016/j.entcom.2022.100491

[5] E. S. de Lima, B. Feijó, and A. L. Furtado, "Hierarchical generation of dynamic and nondeterministic quests in games," in Proc. Int. Conf. Advances in Computer Entertainment Technology (ACE 2014), Article 24, 2014. DOI: 10.1145/2663806.2663833

[6] J. Dormans and S. Bakkes, "Generating missions and spaces for adaptable play experiences," IEEE Trans. Computational Intelligence and AI in Games, vol. 3, no. 3, pp. 216-228, 2011. DOI: 10.1109/TCIAIG.2011.2149523

[7] S. Värtinen, P. Hämäläinen, and C. Guckelsberger, "Generating role-playing game quests with GPT language models," IEEE Trans. Games, vol. 16, no. 1, pp. 127-139, 2024. DOI: 10.1109/TG.2022.3228480

[8] T. Ashby, B. K. Webb, G. Knapp, J. Searle, and N. Fulda, "Personalized quest and dialogue generation in role-playing games: A knowledge graph- and language model-based approach," in Proc. 2023 CHI Conference on Human Factors in Computing Systems, Article 290, 2023. DOI: 10.1145/3544548.3581441

[9] C. P. Ranasingha and N. Arambepola, "Large language models for dynamic game content: Procedural side-quest generation," Int. Conf. Applied and Pure Sciences, University of Kelaniya, Sri Lanka, 2024. [Online]. Available: https://www.researchgate.net/publication/385292792

[10] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in Proc. 36th Annual ACM Symp. User Interface Software and Technology (UIST '23), Article 2, 2023. DOI: 10.1145/3586183.3606763

[11] J. Doran and I. Parberry, "Towards procedural quest generation: A structural analysis of RPG quests," Technical Report LARC-2010-02, University of North Texas, 2010. [Online]. Available: https://ianparberry.com/techreports/LARC-2010-02.pdf

[12] J. Doran and I. Parberry, "A prototype quest generator based on a structural analysis of quests from four MMORPGs," in Proc. 2nd Int. Workshop on Procedural Content Generation in Games, 2011. DOI: 10.1145/2000919.2000920

[13] Qwen Team, "Qwen3.5-122B-A10B," ModelScope, 2025. [Online]. Available: https://modelscope.cn/models/Qwen/Qwen3.5-122B-A10B

[14] Qwen Team, "Qwen3.6-27B," Hugging Face, 2025. [Online]. Available: https://huggingface.co/Qwen/Qwen3.6-27B

[15] OpenAI, "GPT-OSS-120B," Hugging Face, 2025. [Online]. Available: https://huggingface.co/openai/GPT-OSS-120B

[16] Meta AI, "Llama-3.3-70B-Instruct," Hugging Face, 2024. [Online]. Available: https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct

[17] Anthropic, "Claude Opus 4.7," Anthropic Documentation, 2025. [Online]. Available: https://docs.anthropic.com/en/docs/about-claude/models

[18] N. Weissmann, "LLM-Based Quest Generation," GitHub, 2026. [Online]. Available: https://github.com/NielsWeissmann236814/LLM-Based-Quest-Generation-for-Role-Playing-Games